

Food Delivery SQL Business Analysis

Query Documentation • Dataset Validation • SQL Review • GitHub Project Guide

Name: M. Kiran Sai Babu

WS.ID: WSASMP595

Source basis: supplied SQL script, question document, and CSV datasets. Numerical outputs were independently recalculated from the CSV files. Original SQL is retained for reference, with corrections shown where required.

1. Dataset and Validation Summary

File	Rows	Columns
customers.csv	50	4
orders.csv	200	7
restaurants.csv	10	4

Dataset facts: 50 customers, 10 restaurants, 200 orders, and 106 completed orders. Overall average order amount across all orders: **673.78**. Second-highest distinct amount: **1192**. Most frequent completed-order payment method: **Debit Card** (33 orders).

Schema check: customers.csv does not contain no_of_orders; orders.csv does not contain Month_name. These schema facts are reflected in the query review below.

1. Display all completed orders along with customer name and restaurant name.

Validation status: Valid logic; JOIN form is recommended for clarity.

Query explanation

- Filters orders to Completed status.
- Maps customer_id and restaurant_id to readable names.
- Returns one row per completed order.
- IDs were cross-checked against the supplied customer and restaurant tables.

Original SQL / supplied approach

```
SELECT customer_name and restaurant_name via correlated subqueries FROM orders WHERE order_status = 'Completed';
```

Recommended SQL

```
SELECT o.order_id, c.customer_name, r.restaurant_name, o.order_status FROM orders o JOIN customers c ON c.customer_id=o.customer_id JOIN restaurants r ON r.restaurant_id=o.restaurant_id WHERE o.order_status='Completed';
```

Cross-verified output preview

order_id	customer_name	restaurant_name	order_status
4	Nandini Rao	Spice Hub	Completed
5	Tarun Verma	Tandoori Nights	Completed
7	Ananya Rao	Pizza Corner	Completed
8	Abhishek Pandey	Food Palace	Completed
9	Sanjay Gupta	Urban Bites	Completed
10	Preethi Nair	BBQ Nation	Completed
12	Ishita Roy	Royal Kitchen	Completed
13	Keerthi Rao	Royal Kitchen	Completed

Validated result rows: 106.

2. Find the total revenue generated by each restaurant.

Validation status: Valid completed-order revenue calculation.

Query explanation

- SUM(order_amount) calculates restaurant revenue.
- Only completed orders are included.
- Rows are grouped by restaurant.
- Descending order provides the revenue ranking.

Original SQL / supplied approach

```
SELECT restaurant_name, SUM(order_amount), RANK() OVER (ORDER BY SUM(order_amount) DESC) FROM completed orders GROUP BY restaurant_id;
```

Recommended SQL

```
SELECT r.restaurant_name, SUM(o.order_amount) AS total_revenue FROM orders o JOIN restaurants r ON r.restaurant_id=o.restaurant_id WHERE o.order_status='Completed' GROUP BY r.restaurant_id,r.restaurant_name ORDER BY total_revenue DESC;
```

Cross-verified output preview

restaurant_id	restaurant_name	total_revenue	rank
5	Tandoori Nights	14172	1
4	Royal Kitchen	11039	2
3	Urban Bites	9166	3
10	BBQ Nation	8769	4
1	Spice Hub	6538	5
6	Pizza Corner	5864	6
7	Burger House	5076	7
8	Andhra Meals	4732	8

Validated result rows: 10.

3. Show the total number of orders placed by each customer.

Validation status: Valid for completed orders.

Query explanation

- Counts completed orders for each customer.
- Groups by customer_id and name.
- Sorts from highest to lowest order count.
- Customers without completed orders are omitted.

Original SQL / supplied approach

```
SELECT customer_name, COUNT(order_status) FROM completed orders GROUP BY customer_id ORDER BY count DESC;
```

Recommended SQL

```
SELECT c.customer_name, COUNT(o.order_id) AS total_orders FROM customers c JOIN orders o ON o.customer_id=c.customer_id WHERE o.order_status='Completed' GROUP BY c.customer_id,c.customer_name ORDER BY total_orders DESC;
```

Cross-verified output preview

customer_id	customer_name	total_orders
30	Nandini Rao	6
15	Siddharth Jain	6
17	Manoj Kumar	5
35	Sanjay Gupta	5
41	Tarun Verma	5
20	Shreya Menon	4
28	Tanvi Shah	4
11	Aditya Kapoor	3

Validated result rows: 44.

4. Find the average order amount for each city.

Validation status: Valid city-level average calculation.

Query explanation

- Maps orders to restaurant city.
- AVG calculates the city-level mean.
- Only completed orders contribute.
- Values are rounded to two decimals.

Original SQL / supplied approach

```
SELECT city, ROUND(AVG(order_amount),2) FROM completed orders GROUP BY city;
```

Recommended SQL

```
SELECT r.city, ROUND(AVG(o.order_amount),2) AS average_order_amount FROM orders o JOIN restaurants r ON r.restaurant_id=o.restaurant_id WHERE o.order_status='Completed' GROUP BY r.city ORDER BY r.city;
```

Cross-verified output preview

city	average_order_amount
Bangalore	646.30
Delhi	731.43
Hyderabad	647.15
Mumbai	726.44

Validated result rows: 4.

5. Categorize orders as Low, Medium, and High based on order amount.

Validation status: Category logic is valid; FIELD() contains a spelling mismatch for High Category.

Query explanation

- CASE converts amount into a category.
- Below 411.75 is Low.
- Above 934.75 is High.
- All other amounts are Medium.

Original SQL / supplied approach

```
SELECT order_amount, CASE WHEN amount < 411.75 THEN 'Low Category' WHEN amount > 934.75 THEN 'High Category' ELSE 'Medium Category' END;
```

Recommended SQL

```
SELECT order_id, order_amount, CASE WHEN order_amount<411.75 THEN 'Low Category' WHEN order_amount>934.75 THEN 'High Category' ELSE 'Medium Category' END AS order_category FROM orders;
```

Cross-verified output preview

order_id	order_amount	order_category
1	938	High Category
2	173	Low Category
3	378	Low Category
4	689	Medium Category
5	1189	High Category
6	151	Low Category
7	779	Medium Category
8	325	Low Category

Validated result rows: 200.

6. Display restaurants whose average order value is greater than the overall average order value.

Validation status: Runs, but compares completed restaurant averages with an all-order overall average.

Query explanation

- Calculates average completed-order value by restaurant.
- Compares it with the all-order average used in the supplied SQL.
- Labels results Greater AVG or Lesser AVG.
- The two comparison populations differ and this is documented.

Original SQL / supplied approach

```
SELECT restaurant_name, ROUND(AVG(order_amount),2), CASE WHEN AVG(order_amount) > (SELECT AVG(order_amount) FROM orders) THEN 'Greater AVG' ELSE 'Lesser AVG' END FROM completed orders GROUP BY restaurant_id;
```

Recommended SQL

```
SELECT r.restaurant_name, ROUND(AVG(o.order_amount),2) AS average_order_amount FROM orders o JOIN restaurants r ON r.restaurant_id=o.restaurant_id WHERE o.order_status='Completed' GROUP BY r.restaurant_id,r.restaurant_name HAVING AVG(o.order_amount)>(SELECT AVG(order_amount) FROM orders);
```

Cross-verified output preview

restaurant_id	restaurant_name	average_order_amount	average_category
1	Spice Hub	726.44	Greater AVG
2	Food Palace	525.50	Lesser AVG
3	Urban Bites	654.71	Lesser AVG
4	Royal Kitchen	689.94	Greater AVG
5	Tandoori Nights	833.65	Greater AVG
6	Pizza Corner	586.40	Lesser AVG
7	Burger House	634.50	Lesser AVG
8	Andhra Meals	591.50	Lesser AVG

Validated result rows: 10.

7. Find customers who have never placed any orders.

Validation status: Incorrect for the stated requirement; use an anti-join.

Query explanation

- The requirement is an anti-join.
- order_id IS NULL inside orders does not find customers absent from orders.
- LEFT JOIN ... IS NULL or NOT EXISTS is appropriate.
- The validated result checks customer IDs against all orders.

Original SQL / supplied approach

```
SELECT customer_name, order_id FROM orders WHERE order_id IS NULL;
```

Recommended SQL

```
SELECT c.customer_id,c.customer_name FROM customers c LEFT JOIN orders o ON o.customer_id=c.customer_id WHERE o.customer_id IS NULL;
```

Cross-verified output preview

customer_id	customer_name
-------------	---------------

Validated result rows: 0.

8. Show the top 5 customers based on total spending.

Validation status: Valid completed-order spending calculation.

Query explanation

- Sums completed order amounts by customer.
- Sorts total spending descending.
- LIMIT 5 keeps five customers.
- Spending is limited to the supplied dataset.

Original SQL / supplied approach

```
SELECT customer_name, SUM(order_amount) FROM completed orders GROUP BY customer_name ORDER BY total_spendings DESC LIMIT 5;
```

Recommended SQL

```
SELECT c.customer_name,SUM(o.order_amount) AS total_spendings FROM customers c JOIN orders o ON o.customer_id=c.customer_id WHERE o.order_status='Completed' GROUP BY c.customer_id,c.customer_name ORDER BY total_spendings DESC LIMIT 5;
```

Cross-verified output preview

customer_id	customer_name	total_spendings
30	Nandini Rao	4380
15	Siddharth Jain	3738
41	Tarun Verma	3475
28	Tanvi Shah	3284
4	Sneha Iyer	3271

Validated result rows: 5.

9. Find the most frequently used payment method.

Validation status: Valid frequency calculation.

Query explanation

- Groups completed orders by payment method.
- Counts usage frequency.
- Sorts frequency descending.
- The leading method and count were independently recalculated.

Original SQL / supplied approach

```
SELECT payment_method, COUNT(payment_method) FROM completed orders GROUP BY payment_method ORDER BY frequency DESC;
```

Recommended SQL

```
SELECT payment_method,COUNT(*) AS frequency FROM orders WHERE order_status='Completed' GROUP BY payment_method ORDER BY frequency DESC LIMIT 1;
```

Cross-verified output preview

payment_method	frequency
Debit Card	33
UPI	26
Cash	24
Credit Card	23

Validated result rows: 4.

10. Calculate the running total of revenue by order date.

Validation status: Valid running-total approach.

Query explanation

- A CTE aggregates completed revenue by date.
- A window SUM creates the cumulative total.
- Same-date orders are aggregated before the window calculation.
- The output was independently recalculated from orders.csv.

Original SQL / supplied approach

```
WITH daily_revenue AS (SELECT order_date, SUM(order_amount) AS daily_revenue FROM completed orders GROUP BY order_date)
SELECT order_date, daily_revenue, SUM(daily_revenue) OVER (ORDER BY order_date) FROM daily_revenue;
```

Recommended SQL

```
WITH daily_revenue AS (SELECT order_date,SUM(order_amount) AS daily_revenue FROM orders WHERE order_status='Completed' GROUP
BY order_date) SELECT order_date,daily_revenue,SUM(daily_revenue) OVER(ORDER BY order_date ROWS UNBOUNDED PRECEDING) AS
running_total_revenue FROM daily_revenue;
```

Cross-verified output preview

order_date	daily_revenue	running_total_revenue
2025-01-01	948	948
2025-01-02	354	1302
2025-01-03	1491	2793
2025-01-05	729	3522
2025-01-07	180	3702
2025-01-08	2240	5942
2025-01-16	281	6223
2025-01-17	1086	7309

Validated result rows: 75.

11. Rank restaurants based on total revenue generated.

Validation status: Valid revenue ranking.

Query explanation

- Aggregates completed revenue by restaurant.
- RANK orders restaurants by revenue.
- Equal totals receive the same rank.
- The calculation was cross-checked against Q2.

Original SQL / supplied approach

```
SELECT restaurant_name, SUM(order_amount), RANK() OVER (ORDER BY SUM(order_amount) DESC) FROM completed orders GROUP BY restaurant_id;
```

Recommended SQL

```
WITH restaurant_revenue AS (SELECT r.restaurant_name,SUM(o.order_amount) AS total_revenue FROM restaurants r JOIN orders o ON o.restaurant_id=r.restaurant_id WHERE o.order_status='Completed' GROUP BY r.restaurant_id,r.restaurant_name) SELECT restaurant_name,total_revenue,RANK() OVER(ORDER BY total_revenue DESC) AS revenue_rank FROM restaurant_revenue;
```

Cross-verified output preview

restaurant_name	total_revenue	rank
Tandoori Nights	14172	1
Royal Kitchen	11039	2
Urban Bites	9166	3
BBQ Nation	8769	4
Spice Hub	6538	5
Pizza Corner	5864	6
Burger House	5076	7
Andhra Meals	4732	8

Validated result rows: 10.

12. Find the second highest order amount from the orders table.

Validation status: Valid second-highest distinct amount calculation.

Query explanation

- Finds the maximum amount.
- Restricts candidates below that maximum.
- Returns the largest remaining amount.
- This is the second-highest distinct amount.

Original SQL / supplied approach

```
SELECT MAX(order_amount) FROM orders WHERE order_amount < (SELECT MAX(order_amount) FROM orders);
```

Recommended SQL

```
SELECT MAX(order_amount) AS second_highest_order_amount FROM orders WHERE order_amount<(SELECT MAX(order_amount) FROM orders);
```

Cross-verified output preview

second_highest_order_amount
1192

Validated result rows: 1.

13. Find customers who placed more orders than the average number of orders per customer.

Validation status: Requires correction because no_of_orders is absent from customers.csv.

Query explanation

- The stated requirement needs order counts calculated for customers.
- customers.csv has no no_of_orders column.
- A LEFT JOIN preserves customers with zero orders.
- The validated average is calculated across all customers.

Original SQL / supplied approach

```
SELECT customer_name, no_of_orders FROM customers WHERE no_of_orders > (SELECT AVG(no_of_orders) FROM customers);
```

Recommended SQL

```
WITH customer_orders AS (SELECT c.customer_id,c.customer_name,COUNT(o.order_id) AS no_of_orders FROM customers c LEFT JOIN orders o ON o.customer_id=c.customer_id GROUP BY c.customer_id,c.customer_name) SELECT customer_name,no_of_orders FROM customer_orders WHERE no_of_orders>(SELECT AVG(no_of_orders) FROM customer_orders);
```

Cross-verified output preview

customer_id	customer_name	no_of_orders
30	Nandini Rao	8
35	Sanjay Gupta	7
15	Siddharth Jain	7
41	Tarun Verma	7
43	Ajay Singh	6
27	Deepak Choudhary	6
42	Gayathri Iyer	6
16	Ishita Roy	6

Validated result rows: 19.

14. Display the top 3 restaurants in each city based on total orders received.

Validation status: Requires correction because ranking is not partitioned by city.

Query explanation

- The requirement is top three within each city.
- A city-partitioned ranking is required.
- ROW_NUMBER/RANK with PARTITION BY city enforces the boundary.
- The validated output ranks completed orders within each city.

Original SQL / supplied approach

```
SELECT city, restaurant_name, completed order count FROM restaurants/orders ORDER BY order_count DESC;
```

Recommended SQL

```
WITH restaurant_orders AS (SELECT r.restaurant_id,r.restaurant_name,r.city,COUNT(o.order_id) AS order_count FROM restaurants
r LEFT JOIN orders o ON o.restaurant_id=r.restaurant_id AND o.order_status='Completed' GROUP BY
r.restaurant_id,r.restaurant_name,r.city), ranked AS (SELECT *,ROW_NUMBER() OVER(PARTITION BY city ORDER BY order_count
DESC,restaurant_name) AS city_rank FROM restaurant_orders) SELECT city,restaurant_name,order_count,city_rank FROM ranked
WHERE city_rank<=3;
```

Cross-verified output preview

restaurant_id	order_count	restaurant_name	city	city_rank
10	11	BBQ Nation	Bangalore	1
6	10	Pizza Corner	Bangalore	2
2	8	Food Palace	Bangalore	3
5	17	Tandoori Nights	Delhi	1
4	16	Royal Kitchen	Delhi	2
3	14	Urban Bites	Delhi	3
8	8	Andhra Meals	Hyderabad	1
9	5	Cafe Aroma	Hyderabad	2

Validated result rows: 9.

15. Calculate month-wise revenue and show the previous month revenue.

Validation status: Requires correction because Month_name is absent and LAG targets the month label rather than revenue.

Query explanation

- Aggregates completed revenue by calendar month.
- LAG should be applied to monthly revenue.
- order_date is used because Month_name is absent from orders.csv.
- The validated result orders months chronologically.

Original SQL / supplied approach

```
SELECT Month_name, SUM(order_amount), LAG(Month_name) OVER() FROM completed orders GROUP BY Month_name;
```

Recommended SQL

```
WITH monthly_revenue AS (SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,SUM(order_amount) AS monthly_revenue FROM orders WHERE order_status='Completed' GROUP BY DATE_FORMAT(order_date, '%Y-%m')) SELECT month,monthly_revenue,LAG(monthly_revenue) OVER(ORDER BY month) AS previous_month_revenue FROM monthly_revenue;
```

Cross-verified output preview

month	monthly_revenue	previous_month_revenue
2025-01	16343	NULL
2025-02	21376	16343.00
2025-03	19604	21376.00
2025-04	15390	19604.00
2025-05	528	15390.00

Validated result rows: 5.

17. SQL Review — Items Requiring Correction or Clarification

Question	Finding	Treatment
5	FIELD() has 'High Category' while CASE returns 'High Category'.	Correct the spelling or use a numeric CASE sort key.
6	Completed restaurant averages are compared with an all-city average population if a like-for-like comparison is intended.	Use the same population if a like-for-like comparison is intended.
7	order_id IS NULL on orders cannot find customers absent.	Use LEFT JOIN ... IS NULL or NOT EXISTS.
13	no_of_orders is not in customers.csv.	Calculate it from orders.
14	No city-partitioned top-three ranking is implemented.	Use ROW_NUMBER/RANK with PARTITION BY city.
15	Month_name is absent and LAG targets the month label.	Derive month from order_date and LAG monthly_revenue.

18. GitHub Repository Package

Repository name: food-delivery-sql-business-analysis

Description: SQL-based business analysis of a food-delivery dataset covering customer behavior, restaurant performance, revenue analysis, order trends, payment methods, rankings, and time-based revenue metrics.

```
food-delivery-sql-business-analysis/  
├── data/  
│   ├── customers.csv  
│   ├── orders.csv  
│   └── restaurants.csv  
├── sql/  
│   └── Project.sql  
├── documentation/  
│   └── food_delivery_sql_business_analysis_documentation.pdf  
├── README.md  
└── .gitignore
```

README

```
# Food Delivery SQL Business Analysis  
  
SQL business analysis project using customer, restaurant, and order datasets.  
  
## Overview  
The project answers 15 business-oriented SQL questions covering revenue, customer activity, order-value segmentation, restaurant performance, payment behavior, running totals, ranking, and month-over-month revenue.  
  
## SQL Concepts  
- Filtering and grouping  
- SUM, COUNT, AVG, MAX  
- JOINS  
- CASE expressions  
- Subqueries and anti-joins  
- CTEs  
- Window functions  
- RANK, ROW_NUMBER, and LAG  
  
## Validation  
The SQL was reviewed against the supplied CSV schemas and the numerical results were independently recalculated. Several original statements require correction or clarification, including the no-order customer query, top-three-per-city ranking, missing no_of_orders field, and previous-month revenue calculation.  
  
## Structure  
data/ – CSV datasets  
sql/ – SQL scripts  
documentation/ – detailed PDF documentation  
README.md – project overview
```

19. Final Validation Checklist

- ✓ Questions were taken from the supplied question document.
- ✓ All three CSV datasets were inspected and used for validation.
- ✓ Every query has at least four explanation points.
- ✓ Output previews were independently calculated from the datasets.
- ✓ Schema mismatches and logic issues were explicitly documented.
- ✓ Running-total logic aggregates same-date revenue before the window function.
- ✓ Second-highest amount was validated as the second-highest distinct value.
- ✓ GitHub repository name, description, structure, and README were included.